Relatório de Web Crawling - LAB2

Francisco Sousa, João Nuno 53642, 51557 IACD

9 de março de 2026

1 Introdução

A recolha automática de dados na Internet é uma técnica fundamental em áreas como análise de dados, ciência de dados e inteligência artificial. Através de *web scraping* ou utilização de APIs públicas é possível extrair informação atualizada de diferentes plataformas online.

Neste trabalho foi desenvolvido um script em Python que funciona como um pequeno *crawler* financeiro. O objetivo do sistema é recolher automaticamente informação atualizada sobre diferentes ativos financeiros, incluindo ações tecnológicas e criptomoedas, e armazenar esses dados para posterior análise.

O sistema recolhe dados de várias empresas e ativos financeiros, nomeadamente Apple (AAPL), Microsoft (MSFT), Tesla (TSLA), Nvidia (NVDA), Amazon (AMZN) e Bitcoin (BTC-USD).

Os dados recolhidos são posteriormente armazenados num ficheiro CSV, permitindo a construção de históricos e a realização de análises estatísticas ou de séries temporais.

2 Contexto do Site e Fonte de Dados

A informação financeira utilizada neste projeto é obtida a partir do serviço disponibilizado pelo **Yahoo Finance**. Esta plataforma é amplamente utilizada para consulta de informação sobre mercados financeiros, incluindo:

- preços de ações em tempo real
- volume de transações
- capitalização de mercado
- indicadores financeiros
- informação histórica

Em vez de fazer scraping direto do HTML da página web, foi utilizada a biblioteca Python **yfinance**, que funciona como uma interface para os dados do Yahoo Finance.

Esta abordagem apresenta várias vantagens:

- evita parsing complexo de HTML
- reduz a probabilidade de bloqueio pelo website
- simplifica significativamente o código
- permite acesso direto a estruturas de dados organizadas

Desta forma, o script atua como um *data crawler* financeiro, recolhendo periodicamente informação de diferentes ativos.

3 Arquitetura do Script

O sistema desenvolvido segue uma arquitetura simples composta por várias etapas principais:

1. Definição dos ativos financeiros a monitorizar
2. Conexão à API através da biblioteca *yfinance*
3. Extração de dados financeiros relevantes
4. Validação e tratamento de dados
5. Armazenamento em ficheiro CSV
6. Registo de eventos e erros em ficheiro de log

A Figura conceptual abaixo descreve o fluxo do sistema:

Entrada de Ativos → API Yahoo Finance → Extração → Processamento → Armazenamento

O script executa um ciclo que percorre todos os *tickers* definidos e extrai a informação relevante para cada ativo.

3.1 Bibliotecas Utilizadas

Foram utilizadas várias bibliotecas Python para suportar o funcionamento do sistema:

- **yfinance** – acesso aos dados financeiros do Yahoo Finance
- **pandas** – manipulação e armazenamento de dados em formato tabular
- **logging** – registo de eventos e erros do sistema
- **datetime** – registo da data e hora da extração
- **os** – verificação da existência de ficheiros
- **time** – controlo de intervalos entre pedidos

Estas bibliotecas permitem criar um sistema robusto e organizado para recolha automática de dados.

4 Estrutura do Código

O programa principal encontra-se organizado numa função chamada `executar_scraping()`, responsável por coordenar todo o processo de recolha.

Um exemplo simplificado da lógica principal é apresentado abaixo:

```
for ticker_symbol in TICKERS:

    ticker_obj = yf.Ticker(ticker_symbol)

    f_info = ticker_obj.fast_info
    full_info = ticker_obj.info

    preco = f_info['last_price']

    dados = {
```

```
        'ticker': ticker_symbol,
        'preco_atual': preco,
        'volume_diario': f_info['last_volume'],
        'data_extracao': datetime.now()
    }
```

Após a recolha dos dados, estes são convertidos para um *DataFrame* da biblioteca *pandas* e guardados num ficheiro CSV.

5 Atributos Recolhidos

O script recolhe um conjunto de atributos relevantes para cada ativo financeiro.

Os principais atributos extraídos são:

- **ticker** – símbolo do ativo financeiro
- **nome** – nome da empresa ou ativo
- **preço atual** – valor atual de mercado
- **moeda** – moeda utilizada na cotação
- **variação diária (%)** – variação percentual em relação ao fecho anterior
- **máximo do dia** – preço máximo observado no dia
- **mínimo do dia** – preço mínimo observado no dia
- **volume diário** – número de transações
- **market cap** – capitalização de mercado
- **PE ratio** – indicador preço/lucro
- **data de extração** – momento da recolha dos dados

Estes atributos permitem realizar análises financeiras básicas e comparar diferentes ativos.

6 Armazenamento de Dados

Os dados recolhidos são armazenados num ficheiro CSV denominado `monitor_financeiro_v3.csv`. Abaixo apresenta-se uma representação do output gerado:

```
--- Iniciando Captura de Dados: 09/03/2026 15:48:08 ---
✅ AAPL     | Preço: 257.47   | Var: 0.23%
✅ MSFT     | Preço: 408.45   | Var: 0.07%
✅ TSLA     | Preço: 388.09   | Var: -2.01%
✅ BTC-USD  | Preço: 68830.17 | Var: 4.3%
✅ NVDA     | Preço: 179.15   | Var: 0.6%
✅ AMZN     | Preço: 210.08   | Var: -1.32%

========================================
       RELATÓRIO DE ESTATÍSTICAS
========================================
Total de ativos analisados: 6

Top 3 ativos mais caros nesta extração:
- Bitcoin USD (BTC-USD): 68830.17 USD
- Microsoft Corporation (MSFT): 408.45 USD
- Tesla, Inc. (TSLA): 388.09 USD

Os dados foram consolidados em: monitor_financeiro_v3.csv
========================================
```

Figura 1: Relatório de Estatísticas extraído pelo sistema

Caso o ficheiro já exista, os novos dados são adicionados ao final do ficheiro, permitindo criar um histórico de medições ao longo do tempo.

Esta abordagem facilita:

- análise temporal
- construção de datasets
- visualização em ferramentas externas

7 Dificuldades Encontradas

Durante o desenvolvimento do script surgiram alguns desafios técnicos.

7.1 Dados incompletos

Nem sempre todos os atributos estavam disponíveis para determinados ativos. Para resolver este problema foram utilizados métodos como:

- `dict.get()` para evitar erros de chave inexistente
- verificação de valores nulos

7.2 Controlo de erros

- dados temporariamente indisponíveis

Para lidar com estas situações foi utilizado um bloco:

```python
try:
    ...
except Exception as e:
    logging.error(...)
```

7.3 Rate limiting

Para evitar excesso de pedidos ao serviço foi introduzido um pequeno atraso entre pedidos:

```python
time.sleep(1)
```

Esta prática reduz a probabilidade de bloqueio por parte do servidor.

8 Conclusão

Neste trabalho foi desenvolvido um sistema simples de recolha automática de dados financeiros utilizando Python.

O script permite monitorizar múltiplos ativos financeiros, recolher indicadores relevantes e armazenar a informação para posterior análise.

A utilização da biblioteca *yfinance* permitiu simplificar significativamente o processo de recolha de dados, evitando a necessidade de realizar scraping direto de páginas HTML.

Como trabalho futuro, este sistema poderia ser expandido para incluir:

- recolha automática em intervalos regulares
- análise de tendências
- integração com modelos de previsão financeira